

Cocket Nova Development Board Programming Guide C/C++

Release 0.1.0

Cesar Bautista

Jun 26, 2026

CONTENTS

1 Cocket Nova Developmet Board CH552 Guide **3**
- 1.1 Why Use the SDCC Compiler? . . . 3
- 1.2 Understanding Programming Languages in Embedded Systems . . . 3
- 1.3 Requirements . . . 4
- 1.4 PinOut . . . 4

2 PlatformIO Support for CH55x **7**
- 2.1 Requirements . . . 7
- 2.2 Build a PlatformIO Example . . . 8
- 2.3 Upload Firmware . . . 8
- 2.4 PlatformIO Configuration . . . 9
- 2.5 Supported Boards . . . 10
- 2.6 Create a New SDK Project . . . 11

3 Environment Setup on Ubuntu **13**
- 3.1 Linux Configuration for USB . . . 13

4 Environment Setup on Windows **17**
- 4.1 Compiler Installation . . . 17
- 4.2 Alternative Functionality . . . 18

5 General Board Control **21**
- 5.1 Recommended Operating Conditions . . . 21
- 5.2 Voltage Selector . . . 21
- 5.3 JST Connectors . . . 22
- 5.4 Built-In LEDs . . . 23

6 Analog to Digital Converter (ADC) **25**

7 I2C (Inter-Integrated Circuit) **27**
- 7.1 SSD1306 Display . . . 27

8 Interrupts **29**
- 8.1 Timer 0/1 Interrupts . . . 29
- 8.2 External Interrupts . . . 30

9 PWM (Pulse Width Modulation) **31**

10 WS2812 **33**

11 Communication Serial CDC **35**

11.1 USB CDC Serial Configuration for CH55x Microcontrollers 35
11.2 Linux Configuration for USB CDC Devices 37
11.3 Windows Configuration for USB CDC Devices 38

12 How generate report of errors **41**

Note: This project is under active development.

The Coket Nova guide is an instructional manual for utilizing the compiler SDCC. It serves various purposes, allowing users to explore different features and obtain new projects and configurations. The Cocket Nova CH552 boards are incredibly easy to use. The projects focus on innovation and present various alternatives for usage.

Choose one development workflow:

- Use *PlatformIO Support for CH55x* for the supported Ubuntu and Windows setup. This option does not require installing SDCC, Make or the manual toolchain described in the Linux and Windows setup pages. It uses the unit_ch55x_docker_sdk repository as the SDK base.
- Use *Environment Setup on Ubuntu* or *Environment Setup on Windows* only for the legacy manual SDCC and Make workflow. That previous workflow is kept as reference while support moves to the PlatformIO SDK base.

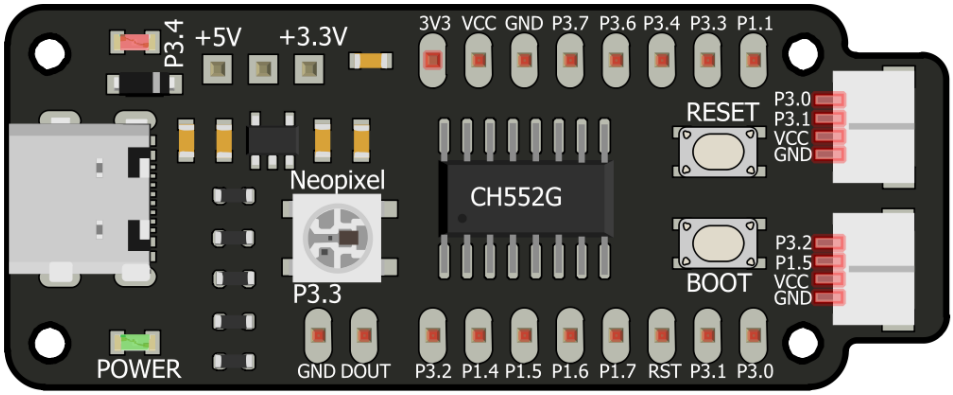


Fig. 1: Cocket Nova CH552 Desing graphic

Tip: The supported SDK examples are available in the unit_ch55x_docker_sdk examples directory. The Blink example is available at examples/blink.

CHAPTER

ONE

COCKET NOVA DEVELOPMET BOARD CH552 GUIDE

Tip: This guide is tailored for individuals with a basic understanding of programming and electronics. It's ideal for those interested in diving into embedded systems and programming microcontrollers.

This is an excellent guide for beginner programmers, focused on using the SDCC compiler in both Windows and Linux environments. Here, you can find excellent references and examples along with comprehensive documentation focusing on developing technology for embedded systems. This course covers everything from installation and setting up the compiler to managing project dependencies and developing code. It's a valuable resource that will guide you through the development process using high-quality technology, ensuring long-lasting and robust projects.

1.1 Why Use the SDCC Compiler?

The Small Device C Compiler (SDCC) is a highly regarded tool in the field of embedded systems development. Here are several reasons why you might choose to use the SDCC compiler for your projects:

1. **Free and Open-Source**: SDCC is freely available and open-source, which means you can use it without licensing fees and contribute to its development if you wish.
2. **Wide Microcontroller Support**: It supports a broad range of microcontrollers, including popular ones like the CH552, making it a versatile choice for various projects.
3. **Ease of Use**: SDCC is known for its user-friendly interface and straightforward setup, which helps developers get started quickly.
4. **Active Community and Documentation**: With an active community and extensive documentation, you can find support and resources to help you solve any issues you encounter.
5. **Compatibility**: SDCC is compatible with many other tools and environments, allowing for seamless integration into existing workflows.

1.2 Understanding Programming Languages in Embedded Systems

To develop effective embedded systems, it's crucial to understand the different types of programming languages used:

1.2.1 Machine Language

Machine language, also known as machine code, is the most fundamental level of programming. Instructions are written in binary bit patterns, which are combinations of 1s and 0s. These patterns correspond to HIGH and LOW voltage levels that the microcontroller or microprocessor can directly interpret. This language is the most difficult for humans to use due to its complexity and lack of readability.

1.2.2 Assembly Language

Assembly language is a step above machine language, providing a more human-readable format. It uses mnemonics and hexadecimal codes to represent machine language instructions. For instance, the 8051 microcontroller assembly language includes a combination of English-like words called mnemonics and hexadecimal numbers. Despite being more readable than machine language, it still requires an in-depth understanding of the microcontroller's architecture.

1.2.3 High-level Language

High-level languages simplify programming by abstracting away the intricate details of the microcontroller's architecture. These languages use familiar words and statements, making them easier to learn and use. Examples of high-level languages include BASIC, C, Pascal, C++, and Java. Programs written in high-level languages are translated into machine code by a compiler, bridging the gap between human-friendly code and machine-understandable instructions.

By understanding these different levels of programming languages , you can choose the most appropriate one for your project, balancing ease of use with the level of control you need over the hardware.

1.3 Requirements

- Python (Package Installation and Environments)
- Utilization of Operating System Controllers
- Understanding of Basic Electronics

1.4 PinOut

The CH552 microcontroller development board has a total of 16 pins, each with a specific function. The following is a list of the pins and their respective functions:

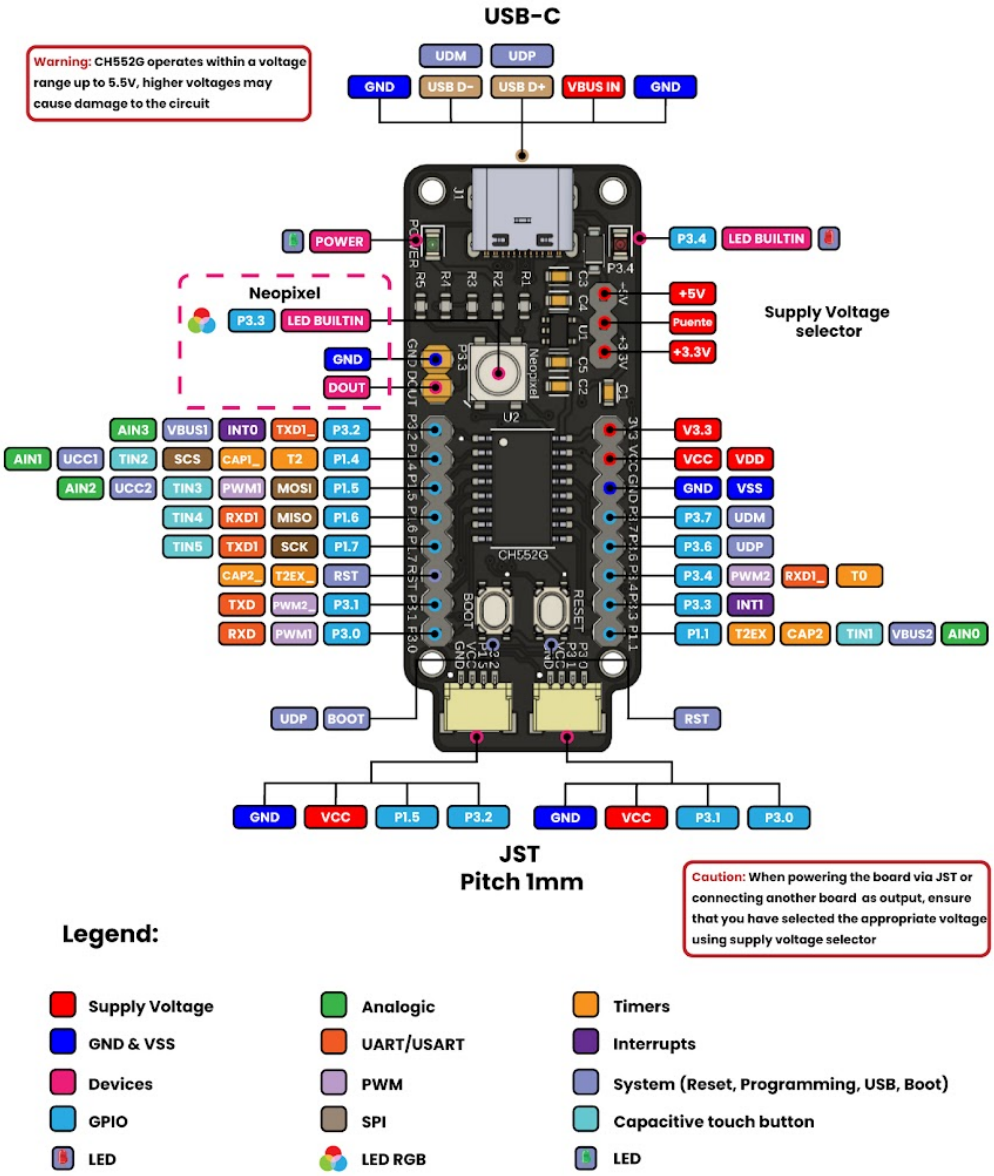


Fig. 1.1: Cocket Nova CH552 PinOut

CHAPTER

TWO

PLATFORMIO SUPPORT FOR CH55X

The unit_ch55x_docker_sdk repository is the SDK base for CH55x PlatformIO projects. It can be used as a local PlatformIO development platform or referenced directly from GitHub. PlatformIO uses SDCC to build the firmware and generates the `firmware.ihx`, `firmware.hex` and `firmware.bin` output files.

This is the supported installation workflow for new Ubuntu and Windows setups. If you use PlatformIO, you do not need to follow the manual SDCC, Make, MinGW or Linux package installation steps from the environment setup pages. Install PlatformIO, configure the project with `platformio.ini` and build with `pio run`.

Note: The previous manual SDCC and Make workflow is kept in this guide as a legacy reference. New support should use the PlatformIO SDK base.

2.1 Requirements

Install PlatformIO Core or use the PlatformIO extension for Visual Studio Code. Git is required to clone the SDK base or to use it directly from `platformio.ini`.

If you want to work with the SDK locally, clone the SDK base repository:

```
git clone git@github.com:UNIT-Electronics-MX/unit_ch55x_docker_sdk.git
cd unit_ch55x_docker_sdk
```

2.1.1 Ubuntu Setup

On Ubuntu, install PlatformIO and Git. The manual SDCC, Make and package setup from the legacy Linux page is not required for PlatformIO projects.

Verify that PlatformIO is available:

```
pio --version
```

For uploads on Ubuntu, the default uploader uses `chprog.py` and requires PyUSB in the PlatformIO Python environment:

```
~/.platformio/penv/bin/python -m pip install pyusb
```

USB access may also require udev permissions:

```
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="4348", ATTR{idProduct}=="55e0", MODE="666"' |␣
↪sudo tee /etc/udev/rules.d/99-ch55x.rules
sudo udevadm control --reload-rules
sudo udevadm trigger
```

2.1.2 Windows Setup

On Windows, install PlatformIO, Git and the CH372 driver. The manual SDCC, MinGW and Make setup from the legacy Windows page is not required for PlatformIO projects.

Install CH372 Driver

Download the latest version of the CH372 driver from the CH372 driver website.

Verify PlatformIO from PowerShell:

```
C:\Users\<user>\.platformio\penv\Scripts\platformio.exe --version
```

Windows upload uses `vnproch55x` when `upload_protocol = auto`.

2.2 Build a PlatformIO Example

From the Blink example inside the SDK repository, run:

```
cd examples/blink
pio run
```

The generated firmware files are placed in the PlatformIO build directory:

```
examples/blink/.pio/build/unit_ch552/
```

The Blink example is available in the SDK base repository: examples/blink.

Examples that include a `platformio.ini` file can be built the same way:

```
cd examples/usb/usb_uart
pio run
```

2.3 Upload Firmware

Put the CH55x board in bootloader mode and run:

```
pio run -t upload
```

The default `upload_protocol = auto` selects `chprog.py` on Linux and macOS, and `vnproch55x` on Windows.

2.3.1 Windows Upload with vnproch55x

On Windows, the Arduino package uses `vnproch55x.exe` from the `devlabtools` package. If PlatformIO does not download the uploader automatically, install it manually in the SDK root:

Note: Edit the paths in the following commands before running them. Replace `C:\path\to\unit_ch55x_docker_sdk` with the folder where you cloned the SDK, and replace `<user>` with your Windows user name.

```
$SdkRoot = "C:\path\to\unit_ch55x_docker_sdk"
cd $SdkRoot
Invoke-WebRequest "https://github.com/UNIT-Electronics/Uelectronics-CH552-Arduino-
↪Package/releases/download/v0.0.6/ch55xduino-tools_mingw32-2026.06.21.tar.bz2" -OutFile
↪"ch55xduino-tools_mingw32-2026.06.21.tar.bz2"
tar -xjf ch55xduino-tools_mingw32-2026.06.21.tar.bz2
Test-Path .\tools\win\vnproch55x.exe
```

The `Test-Path` command must print `True`. The extracted layout must contain:

```
tools/win/vnproch55x.exe
tools/win/*.dll
```

After installing the uploader, build and upload from the PlatformIO project:

```
$PlatformioExe = "C:\Users\<user>\.platformio\penv\Scripts\platformio.exe"
& $PlatformioExe run
& $PlatformioExe run --target upload
```

2.4 PlatformIO Configuration

For a PlatformIO project inside this repository, point `platform` to the SDK root:

```
[env:unit_ch552]
platform = ../..
board = unit_ch552
```

To pin a released SDK version from GitHub, use the release tag in `platformio.ini`:

```
[env:unit_ch552]
platform = https://github.com/UNIT-Electronics-MX/unit_ch55x_docker_sdk.git#v0.1.1
board = unit_ch552
```

To use the current SDK branch directly from GitHub:

```
[env:unit_ch552]
platform = https://github.com/UNIT-Electronics-MX/unit_ch55x_docker_sdk.git
board = unit_ch552
```

For an existing Makefile-style example where `main.c` is located at the project root, set the source directory globally:

```
[platformio]
src_dir = .

[env:unit_ch552]
platform = ../..
board = unit_ch552
```

The default configuration can usually stay minimal:

```
[env:unit_ch552]
platform = ../..
board = unit_ch552

; Arduino defaults to bootcfg 3 for CH552 P3.6 (D+) pull-up.
board_upload.bootcfg = 3
```

You can force a specific uploader when needed:

```
upload_protocol = chprog
upload_protocol = vnproch55x
```

For serial upload:

```
upload_protocol = vnproch55x_serial
upload_port = COM5
```

2.5 Supported Boards

The following board identifiers are available:

Board ID	Target
unit_ch551	CH551
unit_ch552	CH552
unit_ch554	CH554
unit_ch559	CH559

Common build overrides can be added to `platformio.ini`:

```
board_build.f_cpu = 24000000
build_flags =
  -D PIN_LED=P34
```

2.6 Create a New SDK Project

The SDK CLI can create a new project directory:

```
./spkg/spkg init examples/project
```

For the SDK Makefile workflow, the compiled binary is generated at:

```
examples/blink/build/main.bin
```

You can flash that binary using `tools/chprog.py`, wchusbdfu or WCHISPTool.

CHAPTER

THREE

ENVIRONMENT SETUP ON UBUNTU

Caution: This project is under active development. The information provided here is subject to change.

Update the operating system:

```
sudo apt update
```

Install *make*, *binutils* and *sdcc*:

```
sudo apt install make
sudo apt install binutils
sudo apt install sdcc
sudo apt install python3-usb
```

3.1 Linux Configuration for USB

Create a new file:

```
sudo nano /etc/udev/rules.d/99-ch55x.rules
```

Add the following content:

```
SUBSYSTEM=="usb", ATTR{idVendor}=="4348", ATTR{idProduct}=="55e0", MODE="0666"
```

update the rules:

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Clone the SDK repository with the examples:

```
git clone git@github.com:UNIT-Electronics-MX/unit_ch55x_sdk.git
```

Navigate to the path:

```
cd unit_ch55x_sdk/examples/blink
```

Execute the command:

```
make help
```

You will see:

```
Use the following commands:
make all     compile, build, and keep all files
make hex     compile and build blink.hex
make bin     compile and build blink.bin
make clean   remove all build files
```

Connect a device with the BOOT button pressed:

```
lsusb
```

The device will be shown with this description:

```
pc@LAPTOP:~$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 002: ID 4348:55e0 WinChipHead
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

Compile the code:

```
make bin
```

Flash the code, remember to change the device path:

```
sudo python3 ../../tools/chprog.py main.bin
```

Success:

```
pc@LAPTOP:~$ sudo python3 ../../tools/chprog.py main.bin
[sudo] password for pc:
FOUND: CH552 with bootloader v2.4.0.
Erasing chip ...
Flashing main.bin to CH552 ...
SUCCESS: 374 bytes written.
Verifying ...
SUCCESS: 374 bytes verified.
DONE.
```

Note: chprog.py is a python script that allows you to flash the code to the device.

3.1.1 Compile and Flash CH55x with SDCC

3.1.2 Running a Program

To execute the program, open a Bash terminal or **Git Bash terminal on Windows**. Follow these steps to clone the SDK repository, navigate to the Blink example, and compile the project.

1. **Clone the SDK Repository**

   Begin by cloning the SDK repository which contains the examples. Open your bash terminal and execute the following command:

   ```
   git clone git@github.com:UNIT-Electronics-MX/unit_ch55x_sdk.git
   ```

2. **Navigate to the Example Path**

   Once the repository is cloned, navigate to the Blink example:

   ```
   cd unit_ch55x_sdk/examples/blink
   ```

3. **Connect the Device**

   Connect your CH55x device to your computer. Ensure that you press and hold the BOOT button while connecting the device. This is essential for the device to enter programming mode.

4. **Compile the Project**

   To compile the project and generate the necessary files, execute the following command in your terminal:

   ```
   make all
   ```

   This command will compile the project, resulting in the generation of files with various extensions necessary for flashing the microcontroller.

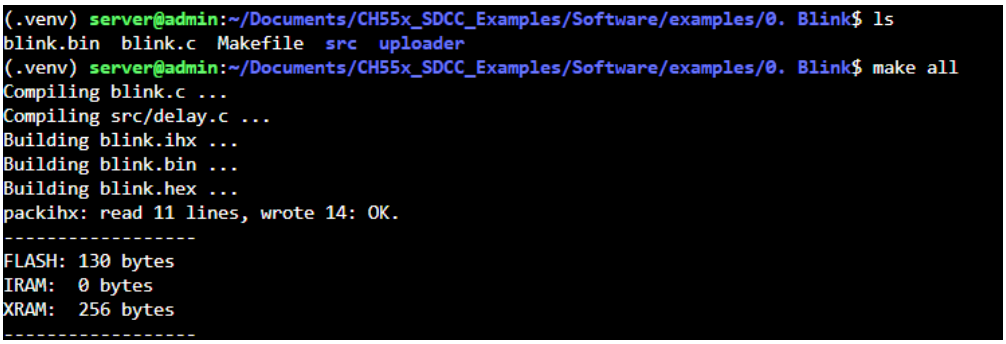


Fig. 3.1: Compilation output files

3.1.3 Install pyusb

pyusb is a Python module necessary for flashing the CH55x microcontroller. To install pyusb, follow these steps:

Install *pyusb* on using pip

```
python3 -m pip install pyusb
```

Then verify the installation

```
python3 -m pip show pyusb
```

For Windows, you can use the following command:

```
pip install pyusb
```

3.1.4 Error with pip

If you encounter this error, we recommend installing the Python environment:

```
sudo apt install python3-venv
```

Create an environment:

```
python3 -m venv .venv
```

Activate the environment:

```
source .venv/bin/activate
```

And install *pyusb*:

```
pip install pyusb
```

CHAPTER

FOUR

ENVIRONMENT SETUP ON WINDOWS

This section provides a step-by-step guide to setting up the SDCC compiler on Windows operating systems. It also includes instructions for installing the necessary tools and configuring the environment variables. Additionally, it covers the installation of the pyusb library and updating the driver using Zadig.

4.1 Compiler Installation

Follow these steps to install the necessary tools and files:

1. **Download or Clone the Repository**

   You can either download the repository as a ZIP file or clone it using Git.

   a. **Clone via Git**

      Install Git for Windows from the official Git website. Then open a command prompt and run:

      ```
      git clone https://github.com/UNIT-Electronics-MX/unit_ch55x_sdk.git
      ```

   b. **Download as ZIP**

      Download the ZIP file from the GitHub repository and extract it to a desired location on your computer.

2. **Install CH372 Driver** Download the latest version of the CH372 driver from the CH372 driver website.
3. **Install Python** Download and install the latest version of Python from the official Python website.
4. **Install Editor Code** Download and install the latest version of your preferred text editor. Some popular choices include Visual Studio Code, Sublime Text, and Notepad++.
5. **Install PlatformIO IDE** Download and install the latest version of Platformio IDE from the visual studio code extensions like in the next image:

Tip: It is recommended to install the tools in the order listed above.

Caution: Remember to restart your computer after installing the tools.

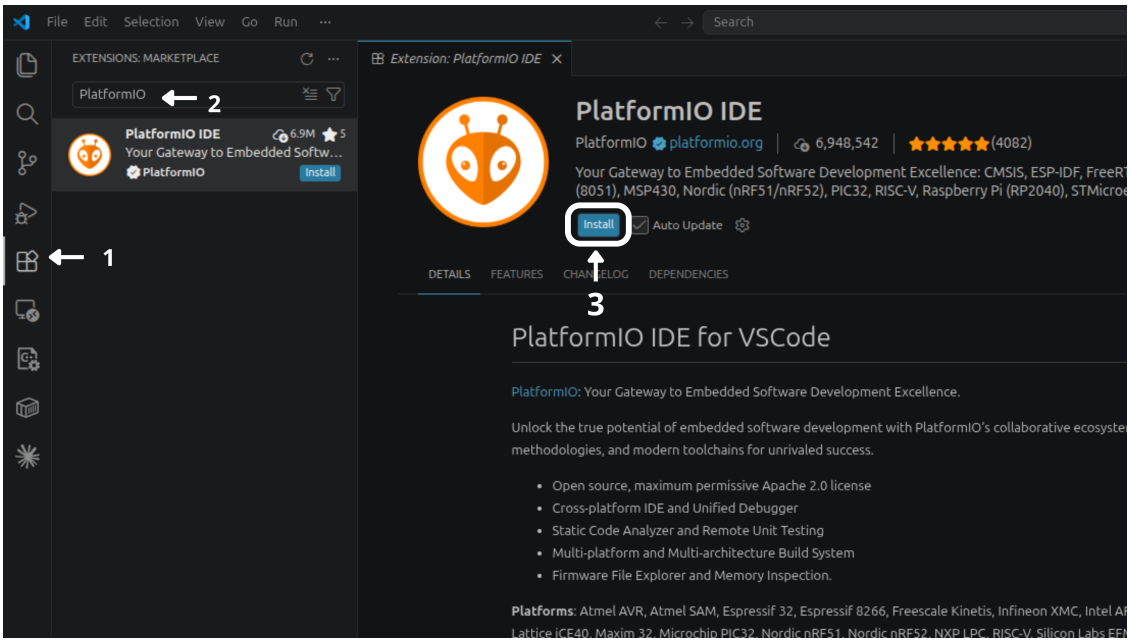


Fig. 4.1: Installing PlatformIO IDE
You can also install PlatformIO IDE from the official Platformio website.

4.2 Alternative Functionality

4.2.1 WCHISPTool

The WCHISPTool is an official programming tool provided by WCH. It is a Windows-based tool that allows users to flash firmware onto CH55x microcontrollers. To use the WCHISPTool, follow these steps:

1. **Download the WCHISPTool**

   Download the WCHISPTool from the official WCH website.

2. **Install the WCHISPTool**

   Install the WCHISPTool on your Windows computer by following the on-screen instructions.

3. **Connect the Device**

   Connect your CH55x device to your computer using a USB cable. Ensure that the BOOT button is pressed while connecting the device.

4. **Flash the Program**

   Open the WCHISPTool and select the appropriate firmware file. Click the "Download" button to flash the program onto the microcontroller.

Note: The WCHISPTool is a Windows-based tool and may not be compatible with other operating systems.

Warning: The WCHISPTool is an official tool provided by WCH and may have limitations compared to other flashing methods.

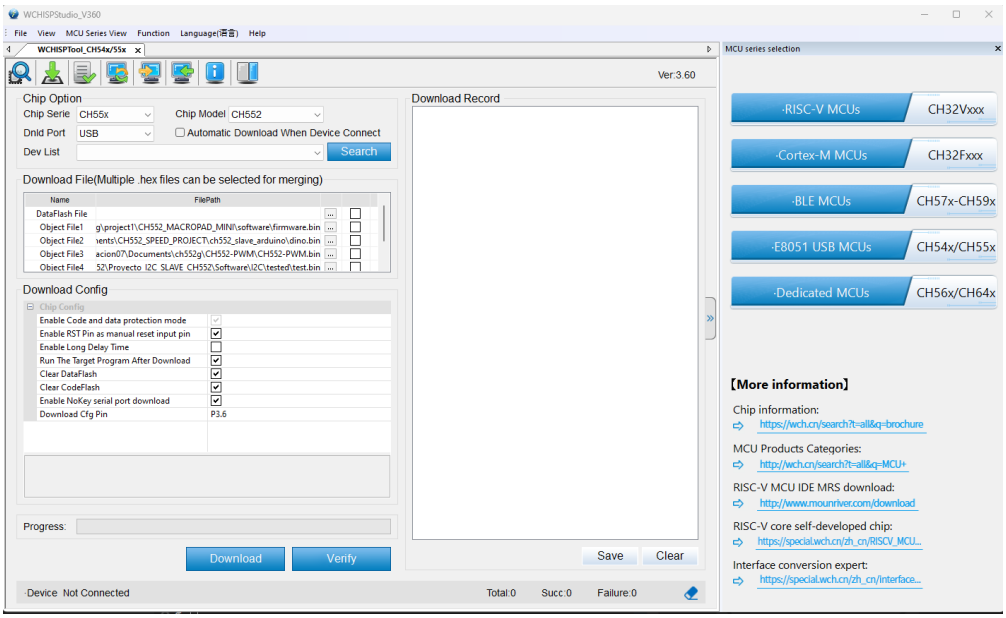


Fig. 4.2: WCHISPTool interface

Note: Requires the ch372 driver to be installed.

CHAPTER

FIVE

GENERAL BOARD CONTROL

The CH552, characterized by its compact size, native USB connectivity, and 16 KB memory (with 14 KB usable), enables the creation of simple yet effective programs. This allows for greater control in implementing various applications. The choice of this microcontroller is based on its affordability, ease of connection, and compatibility with various operating systems.

5.1 Recommended Operating Conditions

Table 5.1: Recommended Operating Conditions

Symbol	Description	Range
VUSB	Voltage supply via USB	3.14 to 5.255 V
VIn	Voltage supply from pins	2.7 to 5.5 V
Top	Operating temperature	-40 to 85 °C

5.2 Voltage Selector

The development board utilizes a clever voltage selector system consisting of three pins and a jumper switch. The configuration of these pins determines the operating voltage of the board. By connecting the central pin to the +5V pin via the jumper, the board operates at 5V. On the other hand, by connecting the central pin to the +3.3V pin, the APK2112K regulator is activated, powering the board at 3.3V. It is crucial to ensure that the jumper switch is in the correct position according to the desired voltage to avoid possible damage to modules, components, and the board itself.

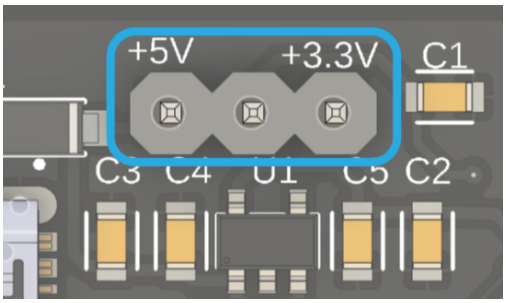


Fig. 5.1: USB Connector

5.3 JST Connectors

The board features two 1mm JST connectors, linked to different pins. The first connector directly connects to GPIO 3.0 and 3.1 of the microcontroller, while the second one is linked to pins 3.2 and 1.5. Both connectors operate in parallel to the selected power supply voltage via the jumper switch. These connectors are compatible with QWIIC, STEMMA QT, or similar pin distribution protocols. It is essential to verify that the selector voltage matches the system voltage to avoid circuit damage. Additionally, these connectors allow the board to be powered and offer functionalities such as PWM and serial communication.

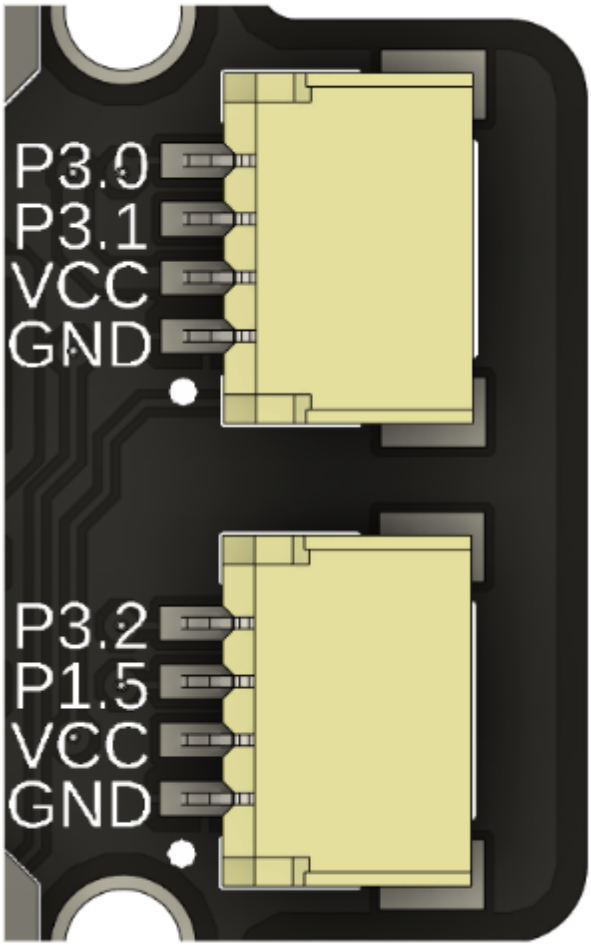


Fig. 5.2: JST Connectors

5.4 Built-In LEDs

The board features two LEDs directly linked to the microcontroller. The first one is connected to pin 3.4, while the second one is a Neopixel LED connected to pin 3.3. This Neopixel provides an output with two headers, one connected to the data output and the other to the board's ground, allowing for the external connection of more LEDs. To use this output, simply connect the DOUT pin to the DIN pin of the next LED in the row. As for power supply, you can use the VCC pin, provided that the external LEDs can operate at this voltage. Otherwise, it will be necessary to power them using an external source.

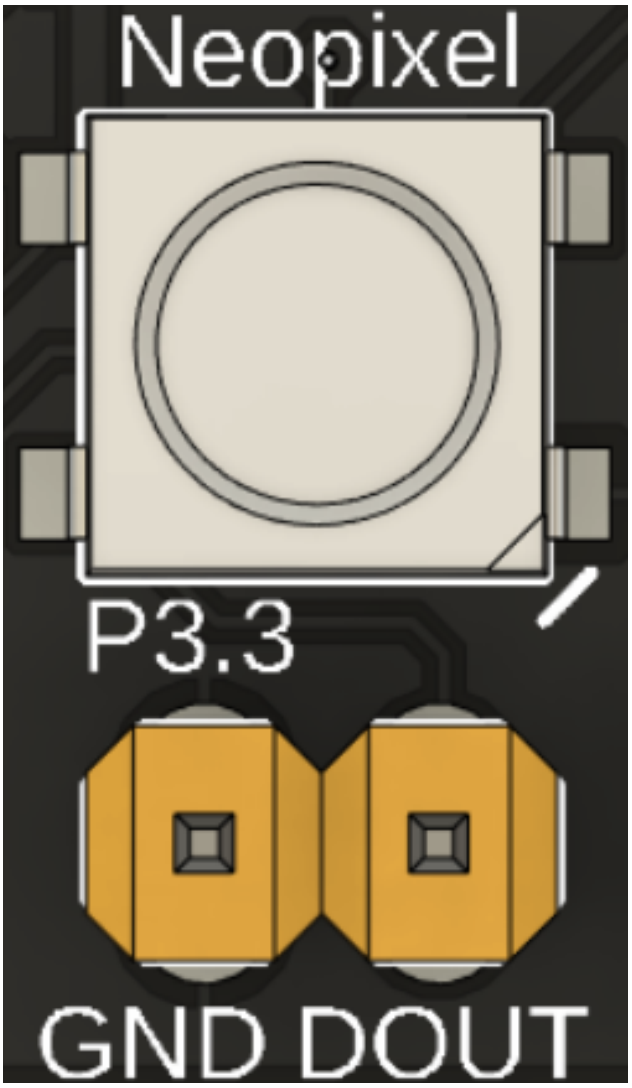


Fig. 5.3: Neopixel LED

CHAPTER

SIX

ANALOG TO DIGITAL CONVERTER (ADC)

The CH552 has four ADC channels, which can be used to read analog values from sensors. The ADC channels are multiplexed with the GPIO pins, so you can use any GPIO pin as an ADC input. the distribution of the ADC channels is as follows.

Table 6.1: Pin Mapping

PIN	ADC Channel
A0	P1.1
A1	P1.4
A2	P1.5
A3	P3.2

The ADC has a resolution of 8 bits, which means it can read values from 0 to 255. The ADC can be configured to read values from 0 to 5V, or from 0 to 3.3V, depending on the VCC voltage:

```
#include "src/config.h" // user configurations
#include "src/system.h" // system functions
#include "src/gpio.h"   // for GPIO
#include "src/delay.h"  // for delays

void main(void)
{
    CLK_config();
    DLY_ms(5);

    ADC_input(PIN_ADC);

    ADC_enable();
    while (1)
    {
        int data = ADC_read(); // Assuming ADC_read() returns an int
    }
}
```

CHAPTER

SEVEN

I2C (INTER-INTEGRATED CIRCUIT)

I2C is a serial communication protocol, so data is transferred bit by bit along a single wire (the SDA line). The SCL line is used to synchronize the data transfer. The I2C protocol is a master-slave protocol, which means that the communication is always initiated by the master device. A master device can communicate with one or multiple slave devices.

Note: The microcontroller use bit banging to communicate with the I2C devices.

All pin configurations are defined in the *config.h* file. The I2C pins are defined as follows

Table 7.1: I2C Pinout

PIN	I2C 1	I2C 2
SDA	P15	P31
SCLK	P32	P30

7.1 SSD1306 Display

The OLED display is a 128x64 pixel monochrome display that uses the I2C protocol for communication.

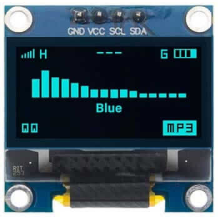


Fig. 7.1: SSD1306 Display

The following code snippet shows how to initialize the OLED display and print a message on the screen:

```
void beep(void) {
uint8_t i;
for(i=255; i; i--) {
    PIN_low(PIN_BUZZER);
    DLY_us(125);
    PIN_high(PIN_BUZZER);
    DLY_us(125);
}
}

void main(void) {
// Setup
CLK_config();                           // configure system clock
DLY_ms(5);                              // wait for clock to stabilize

OLED_init();                            // init OLED

OLED_print("*  UNITelectronics  *");
OLED_print("----------------------\n");
OLED_print("Ready\n");
beep();
while(1) {

}
}
```

Tip: Remember to includes library corresponding to the OLED display in the file.

CHAPTER

EIGHT

INTERRUPTS

The CH552 microcontroller supports 14 sets of interrupt signal sources. These include 6 sets of interrupts (INT0, T0, INT1, T1, UART0, and T2), which are compatible with the standard MCS51, and 8 sets of extended interrupts (SPI0, TKEY, USB, ADC, UART1, PWMX, GPIO, and WDOG). The GPIO interrupt can be chosen from 7 I/O pins.

Table 8.1: Default priority sequence of interrupt sources

Interrupt src.	Entry addr.	Inter-rupt #	Description	Default prio. seq.
INT_NO_INT0	0x0003	0	External interrupt 0	High priority
INT_NO_TMR0	0x000B	1	Timer 0 interrupt	↓
INT_NO_INT1	0x0013	2	External interrupt 1	↓
INT_NO_TMR1	0x001B	3	Timer 1 interrupt	↓
INT_NO_UART0	0x0023	4	UART0 interrupt	↓
INT_NO_TMR2	0x002B	5	Timer 2 interrupt	↓
INT_NO_SPI0	0x0033	6	SPI0 interrupt	↓
INT_NO_TKEY	0x003B	7	Touch key timer interrupt	↓
INT_NO_USB	0x0043	8	USB interrupt	↓
INT_NO_ADC	0x004B	9	ADC interrupt	↓
INT_NO_UART1	0x0053	10	UART1 interrupt	↓
INT_NO_PWMX	0x005B	11	PWM1/PWM2 interrupt	↓
INT_NO_GPIO	0x0063	12	GPIO Interrupt	↓
INT_NO_WDOG	0x006B	13	Watchdog timer interrupt	Low priority

The interrupt priority is determined by the interrupt number.

8.1 Timer 0/1 Interrupts

Timer0 and Timer1 are 16-bit timers/counters controlled by TCON and TMOD. TCON is responsible for timer/counter T0 and T1 startup control, overflow interrupt, and external interrupt control. Each timer consists of dual 8-bit registers forming a 16-bit timing unit. Timer 0's high byte counter is TH0, and its low byte counter is TL0. Similarly, Timer 1's high byte counter is TH1, and its low byte counter is TL1. Timer 1 can also serve as the baud rate generator for UART0. code example:

```
void timer0_interrupt(void) __interrupt(INT_NO_TMR0)        /* Timer0 interrupt service␣
↪routine (ISR) */
{
    PIN_toggle(PIN_BUZZER);
    TH0 = 0xFF;             /* 50ms timer value */
```

(continues on next page)

(continued from previous page)

```
    TL0 = 0x00;
}


int main(void)
{
    CLK_config();
    DLY_ms(5);
    PIN_output(PIN_BUZZER);
    EA  = 1;                /* Enable global interrupt */
    ET0 = 1;                /* Enable timer0 interrupt */

    TH0 = 0xFF;             /* 50ms timer value */
    TL0 = 0x00;
    TMOD = 0x01;            /* Timer0 mode1 */
    TR0 = 1;                /* Start timer0 */
    while(1);
}
```

8.2 External Interrupts

INT0 and INT1 are external interrupt input pins. When an external interrupt occurs, the corresponding interrupt service routine is executed. The external interrupt can be triggered by the falling edge, rising edge, or both edges of the external interrupt input signal. The trigger mode is determined by the external interrupt input pin.

code example:

```
void ext0_interrupt(void) __interrupt(INT_NO_INT0)
{
    PIN_toggle(PIN_LED);
}

int main(void)
{
    CLK_config();
    DLY_ms(5);
    PIN_output_OD(PIN_INT);
    PIN_output(PIN_LED);

    EA  = 1;     /* Enable global interrupt */
    EX0 = 1;    // Enable INT0
    IT0 = 1;    // INT0 is edge triggered

    while(1)
    {
        // Do nothing
    }

}
```

CHAPTER

NINE

PWM (PULSE WIDTH MODULATION)

The PWM module is used to generate a PWM signal on a pin. The PWM signal is generated by changing the duty cycle of the signal. The duty cycle is the ratio of the time the signal is high to the total time of the signal. The PWM module can be used to control the brightness of an LED, the speed of a motor, or the position of a servo motor.

The board contain two PWM pins, which are PIN_PWM and PIN_PWM2. The PWM module can be used to generate a PWM signal on these pins:

```
PWM 1 : P30/P15
PWM 2 : P31/P34
```

Some of the functions provided by the PWM module are:

```
#define MIN_COUNTER 10
#define MAX_COUNTER 254
#define STEP_SIZE   10

void change_pwm(int hex_value)
{
    PWM_write(PIN_PWM, hex_value);
}
void main(void)
{
    CLK_config();
    DLY_ms(5);
    PWM_set_freq(1);
    PIN_output(PIN_PWM);
    PWM_start(PIN_PWM);
    PWM_write(PIN_PWM, 0);
while (1)
{
    for (int i = MIN_COUNTER; i < MAX_COUNTER; i+=STEP_SIZE)
    {
        change_pwm(i);
        DLY_ms(20);
    }
    for (int i = MAX_COUNTER; i > MIN_COUNTER; i-=STEP_SIZE)
    {
        change_pwm(i);
        DLY_ms(20);
    }
}
```

(continues on next page)

(continued from previous page)

```
}
```

CHAPTER

TEN

WS2812

This is a simple library for controlling WS2812 LEDs with an CH552 microcontroller. It is based on the WS2812 datasheet and the CH552 datasheet.

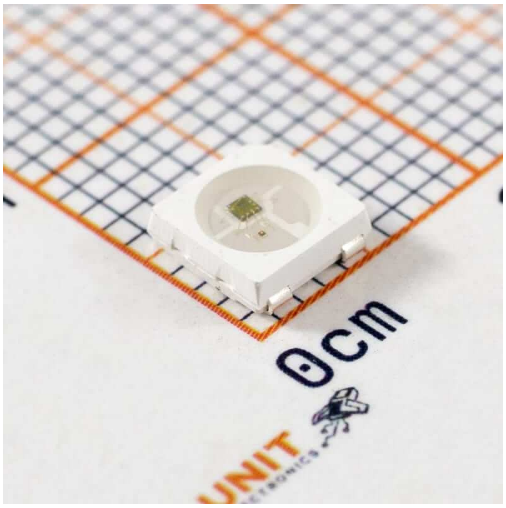


Fig. 10.1: WS2812 LED Strip

Table 10.1: Pin Mapping for WS2812

PIN	GPIO CH552
DOUT	P3.3

```
#define delay 100
#define NeoPixel 16 // Number Neopixel conect
#define level 100 // Ilumination level 0 to 255

void randomColorSequence(void) {

for(int j=0;j<NeoPixel;j++){
    uint8_t red = rand() % level;
    uint8_t green = rand() % level;
    uint8_t blue = rand() % level;
    uint8_t num = rand() % NeoPixel;
```

(continues on next page)

(continued from previous page)

```
    for(int i=0; i<num; i++){
        NEO_writeColor(0, 0, 0);
    }
    NEO_writeColor(red, green, blue);
    DLY_ms(delay);
    NEO_writeColor(0, 0, 0);
    }

    for(int l=0; l<9; l++){
        NEO_writeColor(0, 0, 0);
    }

}

void colorSequence(void) {

for(int j=0;j<=NeoPixel;j++){
        uint8_t red = rand() % level;
        uint8_t green = rand() % level;
        uint8_t blue = rand() % level;
    for(int i=0; i<j; i++){
        NEO_writeColor(red, green, blue);
    }
    DLY_ms(delay);
    for(int l=0; l<j; l++){
        NEO_writeColor(0, 0, 0);
    }
}
}

// ============================================================================
// Main Function
// ============================================================================
void main(void) {
NEO_init();                         // init NeoPixels
CLK_config();                       // configure system clock
DLY_ms(delay);                         // wait for clock to settle

// Loop
while (1) {
    randomColorSequence();
    DLY_ms(100);
    colorSequence();
    DLY_ms(100);
}
}
```

CHAPTER

ELEVEN

COMMUNICATION SERIAL CDC

Cocket Nova development board is compatible with the USB CDC (Communication Device Class) protocol. This allows the Cocket Nova to be used as a virtual serial port. The CDC protocol is supported by most operating systems, including Windows, Linux, and macOS.

Note: The CDC protocol is implemented using the USB peripheral of the microcontroller. The USB peripheral is configured as a virtual serial port, which allows the microcontroller to communicate with the host computer using the USB cable.

The following code snippet shows how to configure the USB peripheral as a virtual serial port and this method only recive information from the host

```
void main(void) {
// Setup
CLK_config();                          // configure system clock
DLY_ms(5);                             // wait for clock to stabilize
CDC_init();                            // init USB CDC

// Loop
while(1) {
    if(CDC_available()) {                // something coming in?
    char c = CDC_read();               // read the character ...
    CDC_writeflush(c);                 // ... and send it back
    }
}
}
```

11.1 USB CDC Serial Configuration for CH55x Microcontrollers

11.1.1 USB Passthrough for CH55x Microcontrollers

This project implements a simple USB passthrough functionality using CH551, CH552, or CH554 microcontrollers. The microcontroller acts as a USB Communication Device Class (CDC), enabling serial communication over USB. Data received via USB is immediately sent back to the host computer. Wiring

Connect the CH55x development board to your PC via USB. It should be automatically detected as a CDC device. Compilation Instructions:

```
Chip: CH551, CH552, or CH554
Clock: 16 MHz internal
Adjust firmware parameters in src/config.h if necessary.
Ensure SDCC toolchain and Python3 with PyUSB are installed.
Press the BOOT button on the board and keep it pressed while connecting it via USB to␣
↪your PC.
Run make flash immediately afterwards to flash the firmware.
For compilation using Arduino IDE, refer to instructions in the .ino file.
```

11.1.2 USB CDC Serial to UART Bridge for CH55x Microcontrollers

This project implements a USB CDC to UART bridge functionality using ch552. The microcontroller acts as a USB Communication Device Class (CDC), allowing serial communication over USB. Data received via USB is sent to the UART interface and vice versa.

```
// Prototypes for used interrupts
void USB_interrupt(void);
void USB_ISR(void) __interrupt(INT_NO_USB) {
USB_interrupt();
}

void UART_interrupt(void);
void UART_ISR(void) __interrupt(INT_NO_UART0) {
UART_interrupt();
}
// ============================================================================
// Main Function
// ============================================================================
void main(void) {
// Setup
CLK_config();                           // configure system clock
DLY_ms(10);                             // wait for clock to settle
UART_init();                            // init UART
CDC_init();                             // init virtual COM

// Loop
while(1) {
    // Handle virtual COM
    if(CDC_available() && UART_ready()) UART_write(CDC_read());
    if(UART_available() && CDC_getDTR()) {
    while(UART_available()) CDC_write(UART_read());
    CDC_flush();
    }
}
}
```

11.1.3 USB CDC PWM Controller for CH55x Microcontrollers

This project implements a USB CDC controlled PWM functionality using CH551, CH552, or CH554 microcontrollers. The microcontroller acts as a USB Communication Device Class (CDC), allowing serial communication over USB. Data received via USB is used to set the PWM value (0-255). Wiring

Connect the CH55x development board to your PC via USB. It should be automatically detected as a CDC device. Compilation Instructions:

```
Chip: CH551, CH552, or CH554
Clock: 16 MHz internal
Adjust firmware parameters in src/config.h if necessary.
Ensure SDCC toolchain and Python3 with PyUSB are installed.
Press the BOOT button on the board and keep it pressed while connecting it via USB to␣
↪your PC.
Run make flash immediately afterwards to flash the firmware.
For compilation using Arduino IDE, refer to instructions in the .ino file.
```

11.2 Linux Configuration for USB CDC Devices

To configure permissions for USB CDC devices (/dev/ttyACM0), follow these steps:

Create a new udev rule file:

```
sudo nano /etc/udev/rules.d/99-custom-usb.rules
```

Add the following rule to the file (replace idVendor and idProduct with your device's actual IDs):

```
SUBSYSTEM=="tty", ATTRS{idVendor}=="1209", ATTRS{idProduct}=="27dd", GROUP="dialout",␣
↪MODE="0666"
```

Save the file (Ctrl + O in nano, then Enter) and exit nano (Ctrl + X).

Reload udev rules for changes to take effect:

```
sudo udevadm control --reload-rules
```

11.2.1 Example Commands for Serial Communication

Send data to USB device:

```
echo -e 'Hello World!\n' > /dev/ttyACM0
```

Read data from USB device:

```
cat /dev/ttyACM0
```

These commands allow you to interact with USB CDC devices connected to your Linux system. Adjust the device path (/dev/ttyACM0) as per your setup.

11.3 Windows Configuration for USB CDC Devices

To configure permissions for USB CDC devices in Windows, follow these steps:

1. Identify the device's COM port number in Device Manager.

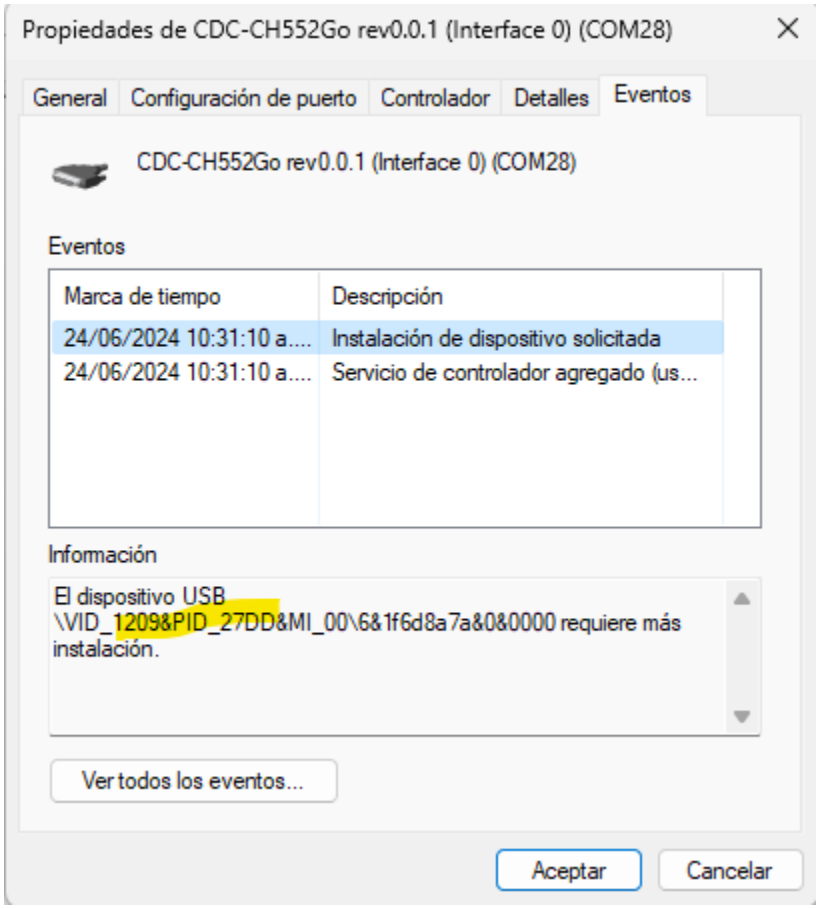


Fig. 11.1: CDC Serial Device Manager

2. Right-click on the device and select Properties.
3. Open Zadig.
4. Go to Options > List All Devices.
5. Select the device from the drop-down list.

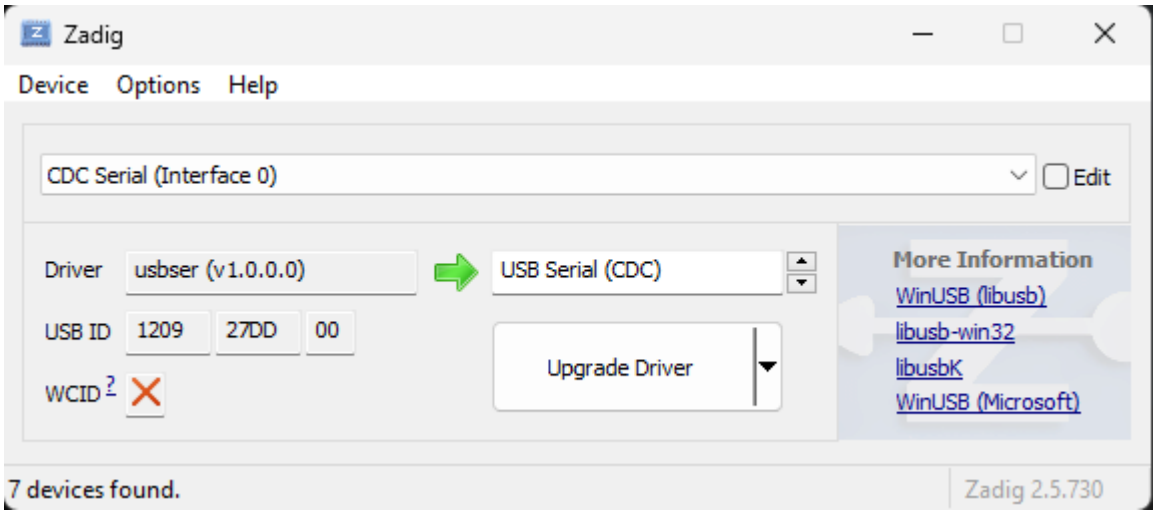


Fig. 11.2: Zadig CDC Device Selection

CHAPTER

TWELVE

HOW GENERATE REPORT OF ERRORS

This is a guide to generate report of errors.

UNIT Electronics is a Mexican-based store that specializes in the provision of electronic components and distinctive electronics solutions. Their primary focus is on the quality of the devices they design.

You can get the report of errors by following the steps below:

1. Got to the Unit Electronics github repository.
2. Click on the issues tab.
3. Click on the new issue button.
4. Fill in the title and description of the issue.
5. Click on the submit button.

The quality control department will review the issue and take the necessary action to resolve it.